Lab Sheet – 7 Full Stack
Khushi Choudhary CU240251564

Setup: Database & Initial Data (SQL)

```
CREATE DATABASE college_db;
USE college_db;
CREATE TABLE departments (
  dept_id  INT PRIMARY KEY,
  dept_name VARCHAR(50) NOT NULL UNIQUE
);
CREATE TABLE students (
  student_id INT PRIMARY KEY,
  name     VARCHAR(50) NOT NULL,
  age      INT CHECK (age >= 17),
  city     VARCHAR(50),
  dept_id   INT,
  FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)
);
CREATE TABLE courses (
  course_id INT PRIMARY KEY,
  title    VARCHAR(80) NOT NULL,
  credits  INT DEFAULT 3,
  dept_id  INT,
  FOREIGN KEY (dept_id) REFERENCES
departments(dept_id)
);
CREATE TABLE enrollments (
  student_id INT,
  course_id  INT,
  marks     INT,
  PRIMARY KEY (student_id, course_id),
  FOREIGN KEY (student_id) REFERENCES
students(student_id),
  FOREIGN KEY (course_id)  REFERENCES
courses(course_id)
);
-- Insert Sample Data
INSERT INTO departments VALUES
(1,'CSE'),(2,'ECE'),(3,'Mechanical'),(4,'Civil');
INSERT INTO students VALUES
(101,'Aarav',20,'Roorkee',1),
(102,'Diya',21,'Dehradun',1),
(103,'Kabir',19,'Haridwar',2),
(104,'Meera',22,'Roorkee',1),
(105,'Rohan',20,'Delhi',3),
(106,'Sana',21,'Dehradun',2),
(107,'Vikram',23,'Jaipur',NULL),
(108,'Isha',19,'Roorkee',2);
INSERT INTO courses VALUES
(201,'Data Structures',4,1),
(202,'Operating Systems',4,1),
(203,'Digital Electronics',3,2),
(204,'Thermodynamics',3,3),
(205,'Machine Learning',4,1);
INSERT INTO enrollments VALUES
(101,201,88),(101,202,76),(101,205,91),
(102,201,67),(102,205,72), (103,203,81),
(104,201,95),(104,202,89), (105,204,58),
(106,203,74), (108,203,66);
```

Departments

dept_id	dept_name
1	CSE
2	ECE
3	Mechanical
4	Civil

Courses

course_id	title	credits	dept_id
201	Data Structures	4	1
202	Operating Systems	4	1
203	Digital Electronics	3	2
204	Thermodynamics	3	3
205	Machine Learning	4	1

Students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Delhi	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2

Enrollments

student_id	course_id	marks
101	201	88
101	202	76
101	205	91
102	201	67
102	205	72
103	203	81
104	201	95
104	202	89
105	204	58
106	203	74
108	203	66

Part A: DDL (Data Definition Language)

A1. Create faculty table

```
CREATE TABLE faculty (
  faculty_id INT PRIMARY KEY,
  name     VARCHAR(50) NOT NULL,
  email    VARCHAR(100) UNIQUE,
  salary   DECIMAL(10, 2),
  dept_id  INT,
  FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
);
```

Faculty

faculty_id	name	email	salary	dept_id
empty				

A2. Add phone column

```
ALTER TABLE faculty
ADD phone VARCHAR(15);
```

Faculty

faculty_id	name	email	salary	dept_id	phone
empty					

A3. Rename phone to contact_no

```
ALTER TABLE faculty
RENAME COLUMN phone TO contact_no;
```

Faculty

faculty_id	name	email	salary	dept_id	contact_no
empty					

A4. Drop column contact_no

```
ALTER TABLE faculty
DROP COLUMN contact_no;
```

Faculty

faculty_id	name	email	salary	dept_id
empty				

A5. Remove faculty table & explain differences

```
DROP TABLE faculty;
```

Differences between DROP, TRUNCATE, and DELETE:

- **DELETE (DML):** Deletes specific or all rows from a table based on a WHERE clause. It logs individual row deletions and can be rolled back.
- **TRUNCATE (DDL):** Removes **all rows** from a table quickly by deallocating table pages. The table structure remains intact.
- **DROP (DDL):** Removes the **entire table structure** along with all data, indexes, constraints, and triggers from the database.

Part B: DML (Data Manipulation Language)

B1. Insert new student

```
INSERT INTO students (student_id, name, age, city, dept_id)
VALUES (109, 'Neha', 20, 'Roorkee', 1);
```

Students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Delhi	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2
109	Neha	20	Roorkee	1

B2. Insert student without specifying city

```
INSERT INTO students (student_id, name, age, dept_id)
VALUES (110, 'Amit', 21, 2);
```

Students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Delhi	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2
109	Neha	20	Roorkee	1
110	Amit	21		2

B3. Update city of student 105

```
UPDATE students
SET city = 'Roorkee'
WHERE student_id = 105;
```

Students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2
109	Neha	20	Roorkee	1
110	Amit	21		2

B4. Increase marks by 5 for course 203

```
UPDATE enrollments
SET marks = marks + 5
WHERE course_id = 203;
```

Enrollments

student_id	course_id	marks
101	201	88
101	202	76
101	205	91
102	201	67
102	205	72
103	203	86
104	201	95
104	202	89
105	204	58
106	203	79
108	203	71

B5. Delete test student(s)

```
DELETE FROM students
WHERE student_id IN (109, 110);
```

Students

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2

B6. Insert student with age 15

```
INSERT INTO students (student_id, name, age, city, dept_id)
VALUES (111, 'Karan', 15, 'Delhi', 1);
```

- **Error Message:** Check constraint 'students_chk_1' is violated. (or ERROR: new row for relation "students" violates check constraint "students_check").
- **Explanation:** The insert fails due to the CHECK (age >= 17) constraint defined on the age column, which prevents inserting any student younger than 17.

Part C: Querying with SELECT

C1. Display all columns of all students

SELECT * FROM students;

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
103	Kabir	19	Haridwar	2
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	
108	Isha	19	Roorkee	2

C2. Display name and city of all students

SELECT name, city FROM students;

name	city
Aarav	Roorkee
Diya	Dehradun
Kabir	Haridwar
Meera	Roorkee
Rohan	Roorkee
Sana	Dehradun
Vikram	Jaipur
Isha	Roorkee

C3. List students from Roorkee

SELECT * FROM students
WHERE city = 'Roorkee';

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2

C4. List students aged between 20 and 22

SELECT * FROM students
WHERE age BETWEEN 20 AND 22;

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2

C5. List students from Roorkee or Dehradun

SELECT * FROM students
WHERE city IN ('Roorkee', 'Dehradun');

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
105	Rohan	20	Roorkee	3
106	Sana	21	Dehradun	2
108	Isha	19	Roorkee	2

C6. List students whose name starts with 'A'

SELECT * FROM students
WHERE name LIKE 'A%';

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1

C7. List students with no department assigned

SELECT * FROM students
WHERE dept_id IS NULL;

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	

C8. Display distinct cities

SELECT DISTINCT city FROM students;

city
Roorkee
Dehradun
Haridwar
Jaipur

C9. Sort by age descending, then name ascending

```
SELECT * FROM students
ORDER BY age DESC, name ASC;
```

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1
106	Sana	21	Dehradun	2
101	Aarav	20	Roorkee	1
105	Rohan	20	Roorkee	3
108	Isha	19	Roorkee	2
103	Kabir	19	Haridwar	2

C10. Top 3 oldest students

```
SELECT * FROM students
ORDER BY age DESC
LIMIT 3;
```

student_id	name	age	city	dept_id
107	Vikram	23	Jaipur	
104	Meera	22	Roorkee	1
102	Diya	21	Dehradun	1

C11. Display name and age with aliases

```
SELECT name AS "Student Name", age AS "Age"
FROM students;
```

Student Name	Age
Aarav	20
Diya	21
Kabir	19
Meera	22
Rohan	20
Sana	21
Vikram	23
Isha	19

Part D: Functions and Aggregation

D1. Total number of students

```
SELECT COUNT(*) AS total_students FROM students;
```

total_students
8

D2. Average, min, and max marks

```
SELECT
  AVG(marks) AS average_marks,
  MIN(marks) AS min_marks,
  MAX(marks) AS max_marks
FROM enrollments;
```

average_marks	min_marks	max_marks
79.27272727272727	58	95

D3. Number of students in each city

```
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;
```

city	student_count
Dehradun	2
Haridwar	1
Jaipur	1
Roorkee	4

D4. Average marks for each course

```
SELECT course_id, AVG(marks) AS avg_marks
FROM enrollments
GROUP BY course_id;
```

course_id	avg_marks
201	83.33333333333333
202	82.5
203	78.66666666666667
204	58
205	81.5

D5. Cities with more than one student

```
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

city	student_count
Dehradun	2
Roorkee	4

D6. Total credits offered by each department

```
SELECT dept_id, SUM(credits) AS total_credits
FROM courses
GROUP BY dept_id;
```

dept_id	total_credits
1	12
2	3
3	3

D7. Uppercase names and name length

```
SELECT UPPER(name) AS uppercase_name, LENGTH(name) AS name_length
FROM students;
```

uppercase_name	name_length
AARAV	5
DIYA	4
KABIR	5
MEERA	5
ROHAN	5
SANA	4
VIKRAM	6
ISHA	4

D8. Explanation: WHERE vs. HAVING

WHERE vs. HAVING:

- WHERE filters individual rows **before** any grouping or aggregation takes place. Therefore, it cannot evaluate aggregate functions like SUM(), AVG(), or COUNT().
- HAVING filters group rows **after** GROUP BY and aggregate functions have been calculated.

Part E: Joins

E1. Student name and department name (INNER JOIN)

```
SELECT s.name AS student_name, d.dept_name
FROM students s
INNER JOIN departments d ON s.dept_id = d.dept_id;
```

student_name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Isha	ECE

E2. All students including those without department (LEFT JOIN)

```
SELECT s.name AS student_name, d.dept_name
FROM students s
LEFT JOIN departments d ON s.dept_id = d.dept_id;
```

student_name	dept_name
Aarav	CSE
Diya	CSE
Kabir	ECE
Meera	CSE
Rohan	Mechanical
Sana	ECE
Vikram	
Isha	ECE

E3. All departments including those with no students

```
SELECT d.dept_name, s.name AS student_name
FROM departments d
LEFT JOIN students s ON d.dept_id = s.dept_id;
```

dept_name	student_name
CSE	Aarav
CSE	Diya
CSE	Meera
ECE	Isha
ECE	Kabir
ECE	Sana
Mechanical	Rohan
Civil	

E4. Student name, course title, and marks

```
SELECT s.name AS student_name, c.title AS course_title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

student_name	course_title	marks
Aarav	Data Structures	88
Aarav	Operating Systems	76
Aarav	Machine Learning	91
Diya	Data Structures	67
Diya	Machine Learning	72
Kabir	Digital Electronics	86
Meera	Data Structures	95
Meera	Operating Systems	89
Rohan	Thermodynamics	58
Sana	Digital Electronics	79
Isha	Digital Electronics	71

E5. Students enrolled in 'Data Structures'

```
SELECT s.name
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id
WHERE c.title = 'Data Structures';
```

name
Aarav
Diya
Meera

E6. Students not enrolled in any course

```
SELECT s.name
FROM students s
LEFT JOIN enrollments e ON s.student_id = e.student_id
WHERE e.course_id IS NULL;
```

name
Vikram

E7. Number of students enrolled in each course (including 0 enrollments)

```
SELECT c.course_id, c.title, COUNT(e.student_id) AS enrolled_students
FROM courses c
LEFT JOIN enrollments e ON c.course_id = e.course_id
GROUP BY c.course_id, c.title;
```

course_id	title	enrolled_students
201	Data Structures	3
202	Operating Systems	2
203	Digital Electronics	3
204	Thermodynamics	1
205	Machine Learning	2

E8. Department name, student count, and average age

```
SELECT d.dept_name, COUNT(s.student_id) AS student_count, AVG(s.age) AS avg_age
FROM departments d
LEFT JOIN students s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name;
```

dept_name	student_count	avg_age
CSE	3	21
Civil	0	
ECE	3	19.666666666666668
Mechanical	1	20

Part F: Subqueries

F1. Students older than average age

```
SELECT * FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

student_id	name	age	city	dept_id
102	Diya	21	Dehradun	1
104	Meera	22	Roorkee	1
106	Sana	21	Dehradun	2
107	Vikram	23	Jaipur	

F2. Student(s) with highest marks

```
SELECT s.name, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
WHERE e.marks = (SELECT MAX(marks) FROM enrollments);
```

name	marks
Meera	95

F3. Courses with at least one enrollment

```
SELECT * FROM courses
WHERE course_id IN (SELECT DISTINCT course_id FROM enrollments);
```

course_id	title	credits	dept_id
201	Data Structures	4	1
202	Operating Systems	4	1
203	Digital Electronics	3	2
204	Thermodynamics	3	3
205	Machine Learning	4	1

F4. Students enrolled in more than 2 courses

```
SELECT * FROM students
WHERE student_id IN (
  SELECT student_id
  FROM enrollments
  GROUP BY student_id
  HAVING COUNT(course_id) > 2
);
```

student_id	name	age	city	dept_id
101	Aarav	20	Roorkee	1

F5. Department with highest number of students

```
SELECT dept_id, COUNT(*) AS student_count
FROM students
WHERE dept_id IS NOT NULL
GROUP BY dept_id
ORDER BY student_count DESC
LIMIT 1;
```

dept_id	student_count
2	3

F6. Students scoring above their course average (Correlated Subquery)

```
SELECT e1.student_id, e1.course_id, e1.marks
FROM enrollments e1
WHERE e1.marks > (
  SELECT AVG(e2.marks)
  FROM enrollments e2
  WHERE e2.course_id = e1.course_id
);
```

student_id	course_id	marks
101	201	88
101	205	91
103	203	86
104	201	95
104	202	89
106	203	79

Part G: Indexes, Views and Transactions

G1. Index creation and EXPLAIN

```
CREATE INDEX idx_students_city ON students(city);
EXPLAIN SELECT * FROM students WHERE city = 'Roorkee';
```

addr	opcode	p1	p2	p3	p4	p5	comment
0	Init	0	15	0		0	
1	OpenRead	0	9	0	5	0	
2	OpenRead	1	15	0	k(2,,)	2	
3	String8	0	1	0	Roorkee	0	
4	SeekGE	1	14	1	1	0	
5	IdxGT	1	14	1	1	0	
6	DeferredSeek	1	0	0		0	
7	Column	0	0	2		0	
8	Column	0	1	3		0	
9	Column	0	2	4		0	
10	Column	1	0	5		0	
11	Column	0	4	6		0	
12	ResultRow	2	5	0		0	
13	Next	1	5	1		0	
14	Halt	0	0	0		0	
15	Transaction	0	0	15	5	1	
16	Goto	0	1	0		0	

G2. Create student_marks view

```
CREATE VIEW student_marks AS
SELECT s.name AS student_name, c.title AS course_title, e.marks
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
JOIN courses c ON e.course_id = c.course_id;
```

G3. Query view for marks > 80

```
SELECT * FROM student_marks
WHERE marks > 80;
```

student_name	course_title	marks
Aarav	Data Structures	88
Aarav	Machine Learning	91
Kabir	Digital Electronics	86
Meera	Data Structures	95
Meera	Operating Systems	89

G4. Drop view and index

```
DROP VIEW student_marks;
DROP INDEX idx_students_city ON students;
```

G5. Transaction with ROLLBACK

```
SELECT marks AS initial_marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
BEGIN TRANSACTION;
UPDATE enrollments
SET marks = 100
WHERE student_id = 101 AND course_id = 201;
ROLLBACK;
SELECT marks AS after_rollback_marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
```

initial_marks
88

after_rollback_marks
88

G6. Transaction with COMMIT

```
SELECT marks AS initial_marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
BEGIN TRANSACTION;
UPDATE enrollments
SET marks = 100
WHERE student_id = 101 AND course_id = 201;
COMMIT;
SELECT marks AS after_commit_marks
FROM enrollments
WHERE student_id = 101 AND course_id = 201;
```

initial_marks
88

after_commit_marks
100

Part H: Challenge Problems (Optional)

H1. Rank list of students by total marks

```
SELECT
  s.student_id,
  s.name,
  SUM(e.marks) AS total_marks,
  RANK() OVER (ORDER BY SUM(e.marks) DESC) AS rank_num,
  DENSE_RANK() OVER (ORDER BY SUM(e.marks) DESC) AS dense_rank_num
FROM students s
JOIN enrollments e ON s.student_id = e.student_id
GROUP BY s.student_id, s.name;
```

student_id	name	total_marks	rank_num	dense_rank_num
101	Aarav	267	1	1
104	Meera	184	2	2
102	Diya	139	3	3
103	Kabir	86	4	4
106	Sana	79	5	5
108	Isha	71	6	6
105	Rohan	58	7	7

H2. Assign grade using CASE statement

```
SELECT
  student_id,
  course_id,
  marks,
  CASE
    WHEN marks >= 90 THEN 'A'
    WHEN marks >= 75 THEN 'B'
    WHEN marks >= 60 THEN 'C'
    ELSE 'D'
  END AS grade
FROM enrollments;
```

student_id	course_id	marks	grade
101	201	100	A
101	202	76	B
101	205	91	A
102	201	67	C
102	205	72	C
103	203	86	B
104	201	95	A
104	202	89	B
105	204	58	D
106	203	79	B
108	203	71	C

H3. Pairs of students from the same city (Self Join)

```
SELECT
  s1.name AS student_1,
  s2.name AS student_2,
  s1.city
FROM students s1
JOIN students s2 ON s1.city = s2.city AND s1.student_id < s2.student_id;
```

student_1	student_2	city
Aarav	Meera	Roorkee
Aarav	Rohan	Roorkee
Aarav	Isha	Roorkee
Diya	Sana	Dehradun
Meera	Rohan	Roorkee
Meera	Isha	Roorkee
Rohan	Isha	Roorkee

H4. Second highest marks without using LIMIT

```
SELECT MAX(marks) AS second_highest_marks
FROM enrollments
WHERE marks < (SELECT MAX(marks) FROM enrollments);
```

second_highest_marks
95